


(video-50) Samarth Vohra

Note:- I did not attend offline class

How to start a backend project?

- ① mkdir folder-name (lets say nitin)
- ② open folder nitin in terminal, type
npm init -y
- ③ npm i express nodemon (to install
express &
nodemon)
- ④ touch index.js.
- ⑤ Change code like to start server ↴

```
const express = require('express');  
const app = express();  
const PORT = 8080;  
app.listen(PORT, () => {  
  console.log(`server working at port number ${PORT}`);  
})
```

ye 6 steps Ratt lo.

Middleware :- (we will study express methods today)

middleware refers to functions that are executed during the lifecycle of a request to the server. These functions have access to the request (req), response (res), and the next() function, which they use to pass control to the next middleware function in the stack.

Middleware is used to process requests, perform common tasks, and either send responses or pass control to subsequent middleware functions or route handlers.

Middleware can perform a variety of tasks such as:

Logging requests.

Authenticating users.

Parsing request bodies (e.g., JSON or form data).

Handling errors.

Serving static files.

Let's say you want to buy something from Amazon:-

amazon.in/iphone/buy

↳ if you hit this url, the system will check if you are logged in or not.

Is like checking ka like / login time stamp, etc
ke like, we use middleware.

iphone/buy → ye url par incoming request
aane per pehle subse pehle middleware
chalega to check if the user is logged in
or not, then app.get('iphone/buy', cb)hne

↑
ye chalega.

```
const express = require('express');
const app = express();
app.use((req, res)=>{
  console.log('I am using middleware');
});
const PORT = 8080;
app.listen(PORT, ()=>{
  console.log(`server working at port number ${PORT}`);
})
```

→ l3

↳ agar mai localhost:8080 / nitin / sharma/singh

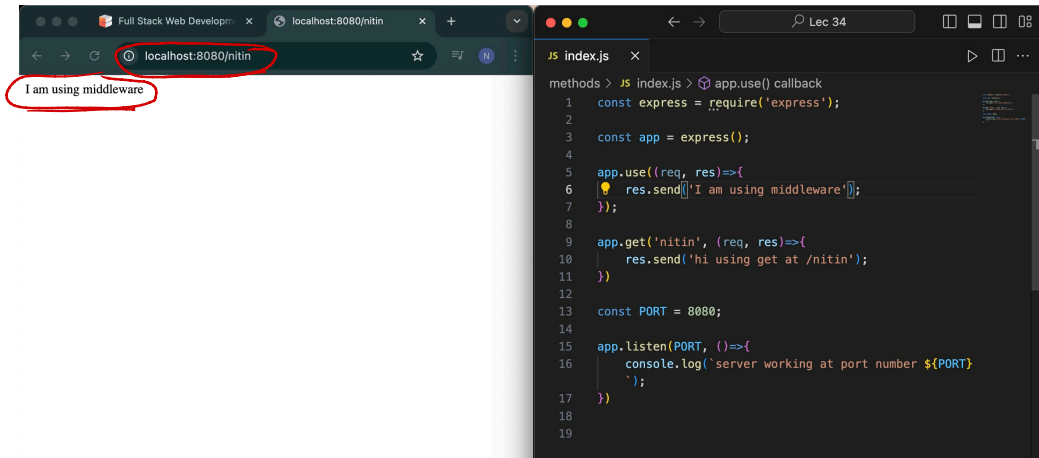
↳ agar mai ye likhunga
tab bhi mera code chalega.
kyu?

middleware ka hamne koi path define nahi kiya,
isliye wo har url par chalega where port is
8080.

No path defined at l3.

Q

```
const express = require('express');
const app = express();
app.use((req, res)=>{
  res.send('I am using middleware');
});
app.get('nitin', (req, res)=>{
  res.send('hi using get at /nitin');
});
const PORT = 8080;
app.listen(PORT, ()=>{
  console.log(`server working at port number ${PORT}`);
})
```



"local host: 8080/ nitin" par bhi middleware function chala kyuki code works from top to bottom.

middleware has no path defined. It will work for all urls.

Let's understand this code:-

```
const express = require('express');

const app = express();

app.use('/nitin', (req, res) => {
  res.send('<h1>I am using middleware</h1>');
});

const PORT = 8080;

app.listen(PORT, () => {
  console.log(`server working at port number ${PORT}`);
})
```

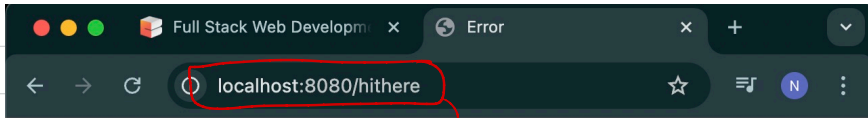


Agar mai

localhost:8080/ Hithere

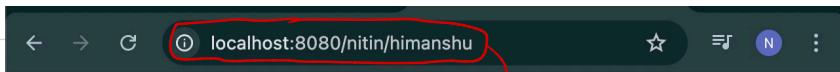
→ ye likhunga tab middleware function nahi chalega kyunki

hamne middleware mei path define kiya hua hai.



→ code mei hamne "/hithere" path mention nahi kiya. ye hai iska matlab.

Agar mai localhost:8080/nitin/sam/aniket search karunga tab kya hoga? Middleware function chal jayega.



I am using middleware

we found that localhost:8080/nitin

/nitin/kumar/singh

/nitin/pooji

ye teeno routes chal jayenge only in case of middleware.

app.use ('/nitin')

→ sirf ye common hona chahiye.

Note: localhost: 8080 / nitinkumar → ye nahi chalega!!

get request (app.get()) :-

```
app.get ( '/samarth' , () => { } );
```

jab bhi koi iss url ko enter karega browser mei,

↓

"localhost: 8080 / samarth"

it will be considered as a request. • Now

jab bhi request aati hai client / user endse, tab do objects aate hai → (req, res)

Server se client / user ko response bhejne ke liye, we use res.send () .

app.get () → it has 2 compulsory parameters

↓

path and cb function

Note:- app.use ('/cat' , cb function);

↓
ye sabhi requests par chalega (get, post, patch, delete, etc) but path same hona chahiye → ('/cat')
Agar path specified nahi hai, then har requests ke har path ke liye chalega.

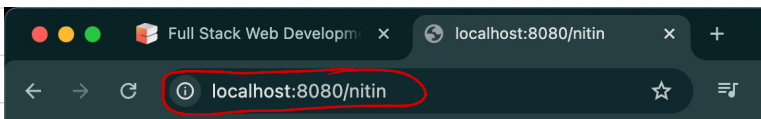
Q

```
const express = require('express');  
const app = express();  
app.use('/nitin', (req, res) => {  
  res.send('<h1>I am using middleware</h1>');  
});  
const PORT = 8080;  
app.get('/nitin', (req, res) => {  
  res.send('<h1>I am using get</h1>');  
});  
app.listen(PORT, () => {  
  console.log(`server working at port number ${PORT}`);  
})
```

l1

l2

agar mai type karu ⇒ localhost:8080/nitin ,
yaha l1 chalega kyunki l1 pehle likha hua
hai ~~and~~ (code works from top to bottom)
and path l1 ka '/nitin' hai.



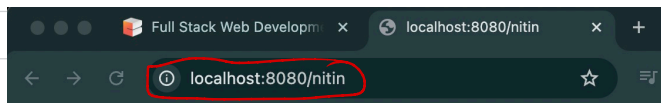
I am using middleware ←

Also, server se sirf ek baar response jata hai, then request cycle khatam ho jati hai.

Agar mujhe l2 wali line chahiye browser par, to kya karenge?

l1 and l2 ko swap kardo ↴

```
const express = require('express');  
  
const app = express();  
  
app.get('/nitin', (req, res) => {  
  res.send('<h1>I am using get</h1>');  
});  
  
app.use('/nitin', (req, res) => {  
  res.send('<h1>I am using middleware</h1>');  
});  
  
const PORT = 8080;  
  
app.listen(PORT, () => {  
  console.log(`server working at port number ${PORT}`);  
})
```



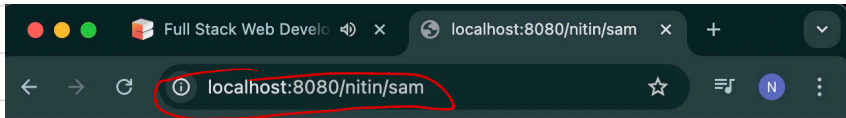
I am using get

To pehle aagya code mei and wone response bhej diya pehle, then uske baad code nahi chalega. Response sirf ek hi baar jaayega.

Agar mei kisi ki localhost: 8080 / nitin / sam ?

ab middleware chalega!

Kyunki get sirf defined path par chalta hai.



I am using middleware ←

default path → '/'

default request → get

Middleware ke andar hum kabhi ~~so~~ bhi response nahi bhejhte ⇒ res.send().

→ we can use it inside middleware but we avoid it.

middleware acts as a middleman. (Bicholiya)

Agar user ne ye url hit kiya → /iphone/bug
then hame ^{check} pchle, karna padega ki user logged

hai ya nahi → ye checking ka kaam middleware karega. Isliye middleware mei response path ko route bhi kehte hai. nahi bhejte.

```

const express = require('express');
const app = express();

app.use('/nitin', (req, res) => {
  console.log('i am using middleware'); → l1
});

app.get('/nitin', (req, res) => {
  res.send('<h1>I am using get</h1>'); l2
});

const PORT = 8080;

app.listen(PORT, () => {
  console.log(`server working at port number ${PORT}`);
})

```

"localhost: 8080/nitin" → ye search karne par kya hoga?

l1 chal jayega but browser will keep waiting for response, it will keep loading. But terminal me l2 console ho jayega.

Code is stuck at l1. Isliye next() use hoga.

next() likhne se code flow will jump to next middleware or function.

```

const express = require('express');
const app = express();

app.use('/nitin', (req, res, next) => {
  console.log('i am using middleware'); l1
  next(); ←
});

app.get('/nitin', (req, res) => {
  res.send('<h1>I am using get</h1>');
});

const PORT = 8080;

app.listen(PORT, () => {
  console.log(`server working at port number ${PORT}`);
})

```

I am using get

```
JS index.js x
methods > JS index.js > ...

79
80 const express = require('express');
81
82 const app = express();
83
84 app.use('/nitin', (req, res, next)=>{
85   console.log('i am using middleware');
86   next();
87 });
88
89 app.get('/nitin', (req, res)=>{
90   res.send('<h1>I am using get</h1>');
91 });
92
93 const PORT = 8080;
94
95 app.listen(PORT, ()=>{
96   console.log('server working at port number ${PORT}');
97 });
98

PROBLEMS TERMINAL ... node - methods + v [] ... ^ x

[nodemon] restarting due to changes...
[nodemon] starting 'node index.js'
server working at port number 8080
i am using middleware
```

→ we can see it console ho gaya.

Q Agar client / user ne random url search kija, then use "404 page not found" dikhana hai. Iske liye kya kare ?

```
const express = require('express');

const app = express();

app.use('/nitin', (req, res, next)=>{
  console.log('i am using middleware');
  next();
});

app.get('/nitin', (req, res)=>{
  res.send('<h1>I am using get</h1>');
});

app.get('*', (req, res)=>{
  res.send('404 page not found');
});

const PORT = 8080;

app.listen(PORT, ()=>{
  console.log('server working at port number ${PORT}');
})
```

] ⇒ it

404 page not found

Check LI on last page, raha hamne
"*" universal selector use kiya taaki random
urls/path/route ke liye LI chal jaaye.

"*" wale code ko sabse neeche likhenge
otherwise har url ke liye * wala code
chalega.

"/nitin" ke liye bhi "*" ka
code chal jayega agar "*" wale
code ko sabse upar likha.

(1:26:00)

Path and Query Parameter :-

https:// www.reddit.com /s/ butterfly

protocol resource URL permanent path temporary path

reddit.com/r/ butterfly

" " " / apple

" " " / football

} → aise kauri
saare anginat
uske honge.

Ab hum har url ke liye unique coding nahi karenge. Hum ek template banayenge jo har url ke liye chal jayegi. Kyunki hum anginat url ke liye unique code nahi likh sakte.

writing

Imagine, `app.get ( )` infinite times for infinite urls. you never create infinite routes.

infinite routes banana isliye bhi possible nahi hai maanlo user is writing "dog" as "dogg" ya "doggy". Tab aap kitne hi routes /url ke liye `app.get()` ka code likh doge.

reddit.com/r/: subreddit

└──┘

↳ path parameter

↓

url ka wo part jo baar
baar change ho raha
hai.

this is the syntax
↓
/r/: subreddit

↓

jaise bhi colon ka use ho, samajh jaana ki path param hai.

Ab mai jab bhi url mai "/r/cat" likhunga tab subreddit ki jagah "cat" aa jayega.

↓

/ r /: sub reddit

So, subreddit is acting like a variable, although subreddit is not a variable.

```
const express = require('express');

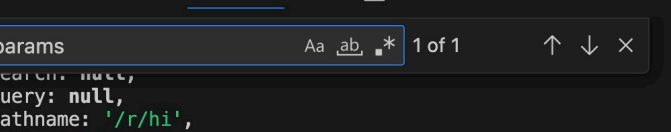
const app = express();

app.get('/r/:machar', (req, res)=>{
  console.log(req);
  res.send('<h1>I am using subreddit</h1>');
});

const PORT = 8080;

app.listen(PORT, ()=>{
  console.log(`server working at port number ${PORT}`);
})
```

↳ we ran this code to find ~~the~~ params and query in req object. check l2.



```
search: null,  
query: null,  
pathname: '/r/hi',  
path: '/r/hi',  
href: '/r/hi',  
_raw: '/r/hi'  
},  
params: { machar: 'hi' },  
query: {},  
res: <ref *3> ServerResponse {  
  _events: [Object: null prototype] { finish: [Function: bou  
nd resOnFinish] },  
  _eventsCount: 1,  
  _maxListeners: undefined,  
  outputData: [],  
  outputSize: 0.
```

maine ye url search kiya tha ↓

http://localhost:8080/r/hi

req object ki key aise the in terminal.

"hi" is in object named params.

object ← params : { machar : 'hi' } → key → value

params and query are objects.

```
const express = require('express');

const app = express();

app.get('/r/:machar', (req, res) => {

  console.log(req.params);

  res.send('<h1>I am using subreddit</h1>');
});

const PORT = 8080;

app.listen(PORT, () => {
  console.log('server working at port number ${PORT}');
})
```

↓ o/p

The screenshot shows a web browser at the top with the address bar containing 'localhost:8080/r/hithere'. Below the browser, there is a red arrow pointing to the text 'I am using subreddit'. To the left of the browser, there is a red arrow pointing to the text '/r/hithere'. Below the browser, there is a red arrow pointing to the text '{ machar: 'hithere' }'. To the right of the browser, there is a red arrow pointing to the text 'this is req.params'. The code editor on the right shows the same code as above, with the output in the terminal window: '[Symbol(kTrailing)]: null, [Symbol(kTrailingCount)]: 0', '(node:mon) restarting due to changes...', '(node:mon) starting node index.js', 'server working at port number 8080', '(node:mon) restarting due to changes...', '(node:mon) starting node index.js', 'server working at port number 8080', '(node:mon) restarting due to changes...', '(node:mon) starting node index.js', 'server working at port number 8080', '{ machar: 'hithere' }'.

Q

```
const express = require('express');

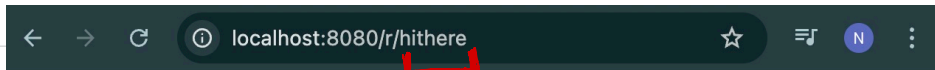
const app = express();

app.get('/r/:machar', (req, res) => {
  let {machar} = req.params; l1
  res.send(req sent at ${machar}); l2
});

const PORT = 8080;

app.listen(PORT, () => {
  console.log('server working at port number ${PORT}');
})
```

let's say I search this $\Rightarrow$ <http://localhost:8080/r/hithere>



req sent at hithere

explanation of l1 $\downarrow$

req. params = { machar : 'hi there' }

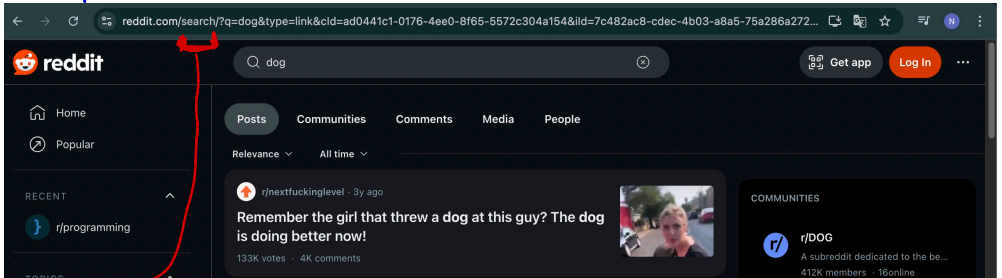
ek variable banaya machar naam ka
jisme wo value hogi jo machar mei
stored hai.

∴ let machar = 'hi there';

उदा. :- temporary path goes to seq. params

Query Parameter :- q → query, q → question

jab bhi question mark dikh jaye samajh jana ki query parameter hai. let's say mai reddit par "dog" search karsta hu.



→ Search is permanent path in the url.

/search / ? q = dogs ...

key value

```

const express = require('express');

const app = express();

app.get('/search', (req, res)=>{
    console.log(req.query);
    res.send('query parameter aa gya');
});

const PORT = 8080;

app.listen(PORT, ()=>{
    console.log(`server working at port number ${PORT}`);
})

```

The screenshot shows a web browser at `localhost:8080/search` displaying the text "query parameter aa gya". To the right, a code editor shows the JavaScript code for the Express.js application. The code defines a GET route for `/search` that logs `req.query` and sends a response. The server is configured to listen on port 8080. Below the code, a terminal window shows the output of running the application with nodemon, confirming that the server is working at port 8080.

empty object

l1 par empty object mila.

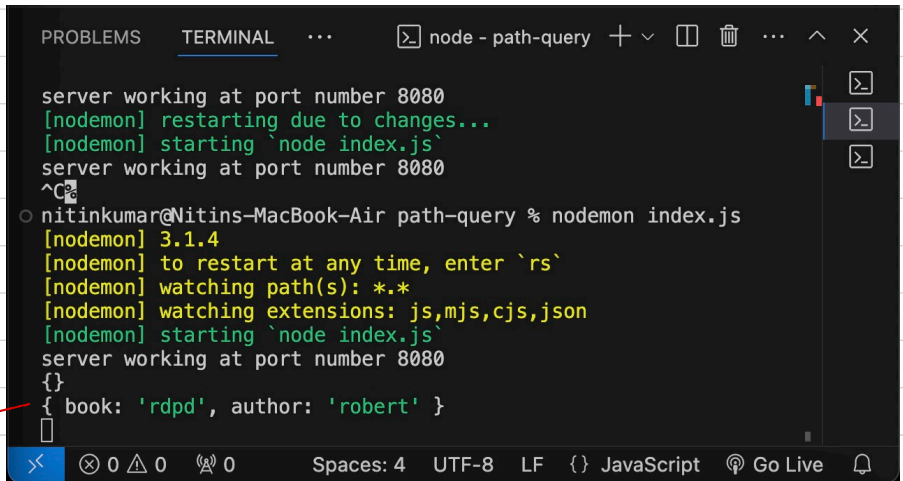
I searched for this url

<http://localhost:8080/search>

question mark and sahi format mei likhne
par ~~ques~~ req. query mei value aa jayegi.

I searched for this url ↴

<http://localhost:8080/search/?book=rdpd&author=robert>



```
PROBLEMS  TERMINAL  ...  node - path-query  + -  [ ]  [ ]  ...  ^  X

server working at port number 8080
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
server working at port number 8080
^C
nitinkumar@Nitins-MacBook-Air path-query % nodemon index.js
[nodemon] 3.1.4
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
server working at port number 8080
{}
{ book: 'rdpd', author: 'robert' }
[ ]

> 0 0 0 0  Spaces: 4  UTF-8  LF  {}  JavaScript  Go Live  [ ]
```

→ this is what I got in ~~req~~ req.query.

req. query = { book: 'rdpd', author: 'robert' }

```
const express = require('express');

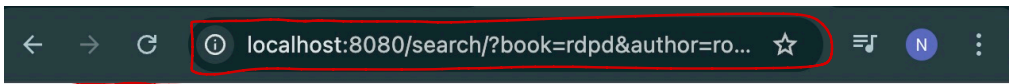
const app = express();

app.get('/search', (req, res) => {

  // console.log(req.query);
  let {book, author} = req.query;
  res.send(`req sent for ${book} & ${author}`);
});

const PORT = 8080;

app.listen(PORT, () => {
  console.log(`server working at port number ${PORT}`);
})
```



req sent for rdpd & robert

`http://localhost:8080/search/?book=rdpd&author=robert`

query parameter → ?

path parameter → :

terminal mei back ke liye ⇒ `cd ..`
↓
space